ЛАБОРАТОРНА РОБОТА № 3
Структурні шаблони

Мета роботи: навчитися реалізовувати структурні шаблони проєктування Адаптер, Декоратор, Міст, Компонувальник, Проксі, Легковаговик

Завдання 1: Адаптер.

1. Створіть клас Logger, який буде мати методи Log(), Error(), Warn(), які виводять повідомлення в консоль різними кольорами (зеленим, червоним і оранжевим відповідно).
2. Створіть клас FileWriter з методами Write(), WriteLine().
3. За допомогою шаблону Адаптер створіть файловий логер.
4. Покажіть правильність роботи свого коду запустивши його в головному методі програми

Лістинг програми:

```
using System;
using System.IO;
using System.Text;

namespace Lab3_Adapter
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.OutputEncoding = Encoding.UTF8;

            Console.WriteLine("Тест Консольного Логера");
            ILogger consoleLogger = new Logger();
            consoleLogger.Log("Повідомлення успішно виведено.");
            consoleLogger.Warn("Увага! Перевірте налаштування.");
            consoleLogger.Error("Помилка! Доступ заборонено.");

            Console.WriteLine();


            Console.WriteLine("Тест Файлового Логера (Адаптер) ");
            FileWriter writer = new FileWriter("log.txt");
            ILogger fileLogger = new FileLoggerAdapter(writer);

            Console.WriteLine("[Запис у файл...] Повідомлення успішно записано у файл
log.txt.");
            fileLogger.Log("все ок.");

            Console.WriteLine("[Запис у файл...] Попередження записано у файл log.txt.");
            fileLogger.Warn("є попередження.");

            Console.WriteLine("[Запис у файл...] Помилка записана у файл log.txt.");
            fileLogger.Error("сталася помилка.");

            Console.WriteLine("\nРоботу завершено. Перевірте файл log.txt.");
            Console.ReadKey();
        }
    }
```

```
    public interface ILogger
    {
        void Log(string message);
        void Error(string message);
        void Warn(string message);
    }

    public class Logger : ILogger
    {
        public void Log(string message)
        {
            Console.ForegroundColor = ConsoleColor.Green;
            Console.WriteLine($"[LOG]: {message}");
            Console.ResetColor();
        }

        public void Error(string message)
        {
            Console.ForegroundColor = ConsoleColor.Red;
            Console.WriteLine($"[ERROR]: {message}");
            Console.ResetColor();
        }

        public void Warn(string message)
        {
            Console.ForegroundColor = ConsoleColor.DarkYellow;
            Console.WriteLine($"[WARN]: {message}");
            Console.ResetColor();
        }
    }

    public class FileWriter
    {
        private string _path;
        public FileWriter(string path) => _path = path;
        public void Write(string text) => File.AppendAllText(_path, text);
        public void WriteLine(string text) => File.AppendAllText(_path, text + Environ-
ment.NewLine);
    }

    public class FileLoggerAdapter : ILogger
    {
        private readonly FileWriter _fileWriter;
        public FileLoggerAdapter(FileWriter fileWriter) => _fileWriter = fileWriter;

        public void Log(string message) => _fileWriter.WriteLine($"[LOG] {DateTime.Now}:
{message}");
        public void Error(string message) => _fileWriter.WriteLine($"[ERROR]
{DateTime.Now}: {message}");
        public void Warn(string message) => _fileWriter.WriteLine($"[WARN]
{DateTime.Now}: {message}");
    }
}
```

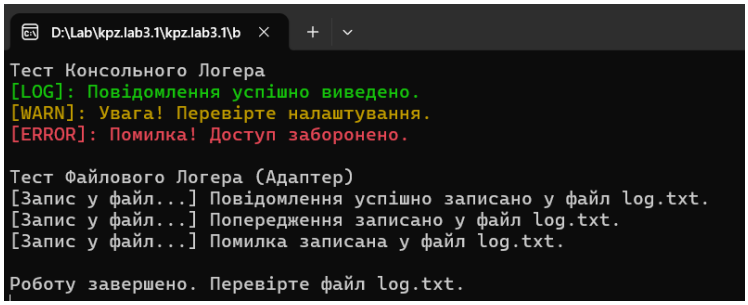

Рис.1.Результат виконання програми:

до бази даних виправити по дизайну.txt log.txt

Файл Редагувати Переглянути H1

```
[LOG] 14.03.2026 9:52:26: Запис у файл: все ок.
[WARN] 14.03.2026 9:52:26: Запис у файл: є попередження.
[ERROR] 14.03.2026 9:52:26: Запис у файл: сталася помилка.
[LOG] 14.03.2026 9:56:27: все ок.
[WARN] 14.03.2026 9:56:27: є попередження.
[ERROR] 14.03.2026 9:56:27: сталася помилка.
[LOG] 14.03.2026 9:59:13: все ок.
[WARN] 14.03.2026 9:59:13: є попередження.
[ERROR] 14.03.2026 9:59:13: сталася помилка.
```

Рис.2.Результат виконання програми:

Завдання 2: Декоратор.

1. Ви розробляєте РПГ гру. Створіть класи героїв Warrior, Mage, Palladin.
2. Для героїв створіть інвентар (одяг, зброю, артефакти), який може підходити будь-якому типу героїв, у вигляді декораторів.
3. Важливою вимогою є можливість використання декількох екземплярів інвентаря на герої одночасно.
4. Покажіть правильність роботи свого коду запустивши його в головному методі програми.

Лістинг програми:

```
using System;
using System.Text;

namespace Lab3_Decorator
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.OutputEncoding = Encoding.UTF8;



            IHero knight = new Knight();
            Console.WriteLine($"\nСтворено: {knight.GetDescription()} (Базова сила:
{knight.GetStats()})");

            knight = new ArmorDecorator(knight, "Важкі лати", 30);
            knight = new WeaponDecorator(knight, "Довгий меч", 15);
            knight = new ArtifactDecorator(knight, "Щит Віри", 20);
            knight = new ArtifactDecorator(knight, "Амулет стійкості", 10);

            Console.WriteLine("\nПерсонаж після екіпірування:");
            Console.WriteLine($"Опис: {knight.GetDescription()}");
            Console.WriteLine($"Загальна сила: {knight.GetStats()}");

            Console.WriteLine("\nТест інших класів ");
            IHero mage = new Mage();
            mage = new WeaponDecorator(mage, "Посох хаосу", 25);
            Console.WriteLine($"{mage.GetDescription()} | Сила: {mage.GetStats()}");

            Console.ReadKey();
        }
    }
```

```
    public interface IHero
    {
        string GetDescription();
        int GetStats();
    }

    public class Warrior : IHero
    {
        public string GetDescription() => "Warrior";
        public int GetStats() => 50;
    }

    public class Mage : IHero
    {
        public string GetDescription() => "Mage";
        public int GetStats() => 30;
    }

    public class Palladin : IHero
    {
        public string GetDescription() => "Palladin";
        public int GetStats() => 45;
    }

    public class Knight : IHero
    {
        public string GetDescription() => "Knight";
        public int GetStats() => 55;
    }

    public abstract class InventoryDecorator : IHero
    {
        protected IHero _hero;
        public InventoryDecorator(IHero hero) => _hero = hero;
        public virtual string GetDescription() => _hero.GetDescription();
        public virtual int GetStats() => _hero.GetStats();
    }

    public class WeaponDecorator : InventoryDecorator
    {
        private string _name;
        private int _bonus;
        public WeaponDecorator(IHero hero, string name, int bonus) : base(hero)
        {
            _name = name;
            _bonus = bonus;
        }
        public override string GetDescription() => base.GetDescription() + $", зі зброєю
[{_name}]";
        public override int GetStats() => base.GetStats() + _bonus;
    }

    public class ArmorDecorator : InventoryDecorator
    {
        private string _name;
        private int _bonus;
        public ArmorDecorator(IHero hero, string name, int bonus) : base(hero)
        {
            _name = name;
            _bonus = bonus;
        }
        public override string GetDescription() => base.GetDescription() + $", в одязі
[{_name}]";
        public override int GetStats() => base.GetStats() + _bonus;
    }
```

```
    public class ArtifactDecorator : InventoryDecorator
    {
        private string _name;
        private int _bonus;
        public ArtifactDecorator(IHero hero, string name, int bonus) : base(hero)
        {
            _name = name;
            _bonus = bonus;
        }
        public override string GetDescription() => base.GetDescription() + $", з
артефактом [{_name}]";
        public override int GetStats() => base.GetStats() + _bonus;
    }
}
```

```
D:\Lab\kpz_lab3.2\kpz_lab3.2\

Створено: Knight (Базова сила: 55)

Персонаж після екіпірування:
Опис: Knight, в одязі [Важкі лати], зі зброєю [Довгий меч], з артефактом [Щит Віри], з артефактом [Амулет стійкості]
Загальна сила: 130

Тест інших класів
Mage, зі зброєю [Посох хаосу] | Сила: 55
```

Рис.3.Результат виконання програми:

Завдання 3: Міст.

1. Ви працюєте над графічним редактором. Створіть базовий клас Shape.
2. Створіть дочірні до Shape класи, Circle, Square, Triangle.
3. За допомогою шаблону Міст додайте можливість рендерингу кожної з фігур як векторної або растрової графіки (вивівши відповідне повідомлення у консоль, наприклад "Drawing Triangle as pixels").
4. Покажіть правильність роботи свого коду запустивши його в головному методі програми.

Лістинг програми:

```
using System;
using System.Text;

namespace Lab3_Bridge
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.OutputEncoding = Encoding.UTF8;

            IRenderer vector = new VectorRenderer();
            IRenderer raster = new RasterRenderer();

            Shape circle = new Circle(vector);
            circle.Draw();

            Shape square = new Square(raster);
            square.Draw();

            Shape triangle = new Triangle(raster);
            triangle.Draw();

            Shape vectorTriangle = new Triangle(vector);
            vectorTriangle.Draw();

            Console.ReadKey();
```

```
        }
    }

    public interface IRenderer
    {
        void RenderShape(string shapeName);
    }

    public class VectorRenderer : IRenderer
    {
        public void RenderShape(string name) => Console.WriteLine($"Drawing {name} as
vectors");
    }

    public class RasterRenderer : IRenderer
    {
        public void RenderShape(string name) => Console.WriteLine($"Drawing {name} as
pixels");
    }

    public abstract class Shape
    {
        protected IRenderer _renderer;
        protected Shape(IRenderer renderer) => _renderer = renderer;
        public abstract void Draw();
    }

    public class Circle : Shape
    {
        public Circle(IRenderer renderer) : base(renderer) { }
        public override void Draw() => _renderer.RenderShape("Circle");
    }

    public class Square : Shape
    {
        public Square(IRenderer renderer) : base(renderer) { }
        public override void Draw() => _renderer.RenderShape("Square");
    }

    public class Triangle : Shape
    {
        public Triangle(IRenderer renderer) : base(renderer) { }
        public override void Draw() => _renderer.RenderShape("Triangle");
    }
}
```

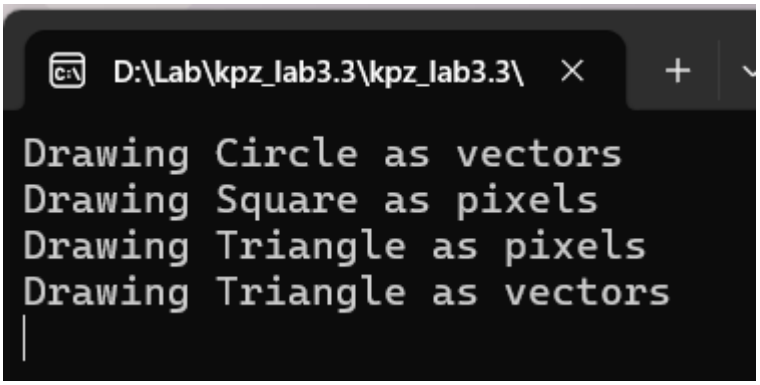

Рис.4.Результат виконання програми:

Завдання 4: Проксі.

1. Створіть клас SmartTextReader, який вміє читати вміст текстового файлу і перетворювати його на двомірний масив якому зовнішній масив відповідає рядкам тексту, а вкладені масиви відповідають символам у відповідному рядку.

2. Створіть проксі для SmartTextReader з логуванням SmartTextChecker, який буде виводити інформацію про успішне відкриття, прочитання і закриття файлу, а також буде виводити загальну кількість рядків і символів у прочитаному тексті.
3. Створіть проксі для SmartTextReader з обмеженням доступу до певних файлів SmartTextReaderLocker. Цей клас в конструкторі приймає регулярний вираз, по якому лімітується доступ до певної групи файлів. Якщо клієнт викликатиме метод для прочитання такого лімітованого файлу, замість прочитання файлу в консоль має виводитися повідомлення “Access denied!”.
4. Покажіть правильність роботи свого коду запустивши його в головному методі програми.
Лістинг програми:

```
using System;
using System.IO;
using System.Text;
using System.Text.RegularExpressions;

namespace Lab3_Proxy
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.OutputEncoding = Encoding.UTF8;

            string testFile = "data.txt";
            string restrictedFile = "admin_secrets.txt";

            File.WriteAllText(testFile, "Рядок один\nРядок два\nСимволи");
            File.WriteAllText(restrictedFile, "Секретні дані");

            ISmartTextReader reader = new SmartTextReader();

            Console.WriteLine("Логуючий проксі");
            ISmartTextReader checker = new SmartTextChecker(reader);
            checker.ReadText(testFile);

            Console.WriteLine("\nОбмеження доступу (Дозволено)");
            ISmartTextReader locker = new SmartTextReaderLocker(reader, @"secrets");
            locker.ReadText(testFile);

            Console.WriteLine("\nОбмеження доступу (Заборонено)");
            locker.ReadText(restrictedFile);

            Console.ReadKey();
        }
    }

    public interface ISmartTextReader
    {
        char[][] ReadText(string path);
    }

    public class SmartTextReader : ISmartTextReader
    {
        public char[][] ReadText(string path)
        {
            string[] lines = File.ReadAllLines(path);
            char[][] result = new char[lines.Length][];
            for (int i = 0; i < lines.Length; i++)
            {
```

```
                result[i] = lines[i].ToCharArray();
            }
            return result;
        }
    }

    public class SmartTextChecker : ISmartTextReader
    {
        private readonly ISmartTextReader _reader;

        public SmartTextChecker(ISmartTextReader reader)
        {
            _reader = reader;
        }

        public char[][] ReadText(string path)
        {
            Console.WriteLine($"Opening file: {path}");
            char[][] data = _reader.ReadText(path);
            Console.WriteLine($"File {path} read successfully.");

            int totalChars = 0;
            foreach (var line in data) totalChars += line.Length;

            Console.WriteLine($"Rows count: {data.Length}");
            Console.WriteLine($"Total characters: {totalChars}");
            Console.WriteLine($"Closing file: {path}");

            return data;
        }
    }

    public class SmartTextReaderLocker : ISmartTextReader
    {
        private readonly ISmartTextReader _reader;
        private readonly Regex _regex;

        public SmartTextReaderLocker(ISmartTextReader reader, string pattern)
        {
            _reader = reader;
            _regex = new Regex(pattern);
        }

        public char[][] ReadText(string path)
        {
            if (_regex.IsMatch(path))
            {
                Console.WriteLine("Access denied!");
                return null;
            }
            return _reader.ReadText(path);
        }
    }
}
```

```
D:\Lab\kpz_lab.3.4\kpz_lab.3.4
Логуючий проксі
Opening file: data.txt
File data.txt read successfully.
Rows count: 3
Total characters: 26
Closing file: data.txt

Обмеження доступу (Дозволено)

Обмеження доступу (Заборонено)
Access denied!
```

Рис.5.Результат виконання програми:

Завдання 5: Компонувальник.

1. Вам потрібно створити власну мову розмітки LightHTML.
2. Кожен елемент розмітки має наслідувати клас LightNode.
3. Створіть два дочірніх класи від LightNode: LightElementNode, LightTextNode.
4. LightTextNode може містити лише текст.
5. LightElementNode може містити будь-які LightNode. LightElementNode повинен мати інформацію про назву тега, його тип відображення (блочний чи рядковий), тип закриття (одиничний тег, як **Помилка! Не вказано ім'я файлу.**чи з закриваючим тегом) список CSS класів, кількість дочірніх елементів, а також має бути можливість виводити на екран його outerHTML і innerHTML.
6. За допомогою своєї мови розмітки виведіть в консоль елемент сторінки на Ваш вибір (наприклад якусь таблицю, список тощо).
7. Покажіть правильність роботи свого коду запустивши його в головному методі програми.

Лістинг програми:

```
using System;
using System.Collections.Generic;
using System.Text;

namespace Lab3_Composite
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.OutputEncoding = Encoding.UTF8;

            var table = new LightElementNode("table", "block", "double");
            table.AddClass("data-table");

            var tr = new LightElementNode("tr", "block", "double");

            var th1 = new LightElementNode("th", "inline", "double");
            th1.AddChild(new LightTextNode("Назва"));

            var th2 = new LightElementNode("th", "inline", "double");
            th2.AddChild(new LightTextNode("Ціна"));

            tr.AddChild(th1);
            tr.AddChild(th2);
```

```
            table.AddChild(tr);

            var row = new LightElementNode("tr", "block", "double");
            var td1 = new LightElementNode("td", "inline", "double");
            td1.AddChild(new LightTextNode("Яблуко"));

            var td2 = new LightElementNode("td", "inline", "double");
            td2.AddChild(new LightTextNode("30 грн"));

            row.AddChild(td1);
            row.AddChild(td2);
            table.AddChild(row);

            var img = new LightElementNode("img", "inline", "single");
            img.AddClass("product-image");
            table.AddChild(img);

            Console.WriteLine("Демонстрація OuterHTML ");
            Console.WriteLine(table.OuterHTML());

            Console.WriteLine("\nДемонстрація InnerHTML ");
            Console.WriteLine(table.InnerHTML());

            Console.ReadKey();
        }
    }

    public abstract class LightNode
    {
        public abstract string OuterHTML();
        public abstract string InnerHTML();
    }

    public class LightTextNode : LightNode
    {
        private string _text;
        public LightTextNode(string text) => _text = text;
        public override string OuterHTML() => _text;
        public override string InnerHTML() => _text;
    }

    public class LightElementNode : LightNode
    {
        private string _tagName;
        private string _displayType;
        private string _closingType;
        private List<string> _classes = new List<string>();
        private List<LightNode> _children = new List<LightNode>();

        public LightElementNode(string tag, string display, string closing)
        {
            _tagName = tag;
            _displayType = display;
            _closingType = closing;
        }

        public void AddClass(string className) => _classes.Add(className);
        public void AddChild(LightNode node) => _children.Add(node);

        public override string InnerHTML()
        {
            StringBuilder sb = new StringBuilder();
            foreach (var child in _children)
            {
                sb.Append(child.OuterHTML());
```

```
            }
            return sb.ToString();
        }

        public override string OuterHTML()
        {
            StringBuilder sb = new StringBuilder();
            sb.Append($"<{_tagName}");

            if (_classes.Count > 0)
            {
                sb.Append($" class=\"{string.Join(" ", _classes)}\"");
            }

            if (_closingType == "single")
            {
                sb.Append(" />");
            }
            else
            {
                sb.Append(">");
                sb.Append(InnerHTML());
                sb.Append($"</{_tagName}>");
            }

            return sb.ToString();
        }
    }
}
```

```
D:\Lab\kpz_lab.3.5\kpz_lab.3.5

Демонстрація OuterHTML
<table class="data-table"><tr><th>Назва</th><th>Ціна</th></tr><tr><td>Яблуко</td><td>30 грн</td></tr><img class="product
-image" /></table>

Демонстрація InnerHTML
<tr><th>Назва</th><th>Ціна</th></tr><tr><td>Яблуко</td><td>30 грн</td></tr><img class="product-image" />
```

Рис.6.Результат виконання програми:

Завдання 6: Легковаговик.

1. За допомогою свого LightHTML з завдання 1 перетворіть текст книги в HTML верстку за такими правилами:

a. Перший рядок має бути елементом

b. Якщо в рядку менше 20 символів - це має бути елемент

c. Якщо рядок починається з пробільного символу - це має бути

d. В будь-якому іншому випадку - елемент

2. Покажіть скільки займає все дерево Вашої верстки, коли воно повністю утримується в пам’яті процесу.

3. Використайте Легковаговик на Ваших класах HTML елементів, щоб зменшити споживання пам’яті.

4. Покажіть правильність роботи свого коду запустивши його в головному методі програми.

Лістинг прогорами:

```
using System;
using System.Collections.Generic;
using System.Text;
```

```
namespace Lab3_Flyweight
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.OutputEncoding = Encoding.UTF8;

            string bookText = "ROMEO AND JULIET\n" +
                              "ACT V\n" +
                              " Scene I. Mantua. A Street.\n" +
                              "The vault is opened and Romeo enters the tomb to find Ju-
liet.";

            string[] lines = bookText.Split('\n');
            var factory = new LightHTMLFactory();
            var body = new LightElementNode("body");

            long startMemory = GC.GetTotalMemory(true);

            for (int i = 0; i < lines.Length; i++)
            {
                string line = lines[i];
                string tagName = "p";

                if (i == 0) tagName = "h1";
                else if (line.StartsWith(" ")) tagName = "blockquote";
                else if (line.Length < 20) tagName = "h2";

                var flyweight = factory.GetElement(tagName);
                var element = new LightElementInstance(flyweight);
                element.AddChild(new LightTextNode(line));
                body.AddChild(element);
            }

            long endMemory = GC.GetTotalMemory(true);

            Console.WriteLine("Результат верстки книги");
            Console.WriteLine(body.OuterHTML());

            Console.WriteLine("\nЛегковаговик");
            Console.WriteLine($"Унікальних об'єктів тегів у пам'яті: {facto-
ry.TotalElements}");
            Console.WriteLine($"Зайнято пам'яті для дерева: {endMemory - startMemory}
байт");

            Console.ReadKey();
        }
    }

    public class LightHTMLElementFlyweight
    {
        public string TagName { get; }
        public LightHTMLElementFlyweight(string tagName) => TagName = tagName;
    }

    public class LightHTMLFactory
    {
        private Dictionary<string, LightHTMLElementFlyweight> _elements = new Diction-
ary<string, LightHTMLElementFlyweight>();

        public LightHTMLElementFlyweight GetElement(string tagName)
        {
            if (!_elements.ContainsKey(tagName))
```

```
                _elements[tagName] = new LightHTMLElementFlyweight(tagName);
            return _elements[tagName];
        }

        public int TotalElements => _elements.Count;
    }

    public abstract class LightNode
    {
        public abstract string OuterHTML();
    }

    public class LightTextNode : LightNode
    {
        private string _text;
        public LightTextNode(string text) => _text = text;
        public override string OuterHTML() => _text;
    }

    public class LightElementInstance : LightNode
    {
        private LightHTMLElementFlyweight _flyweight;
        private List<LightNode> _children = new List<LightNode>();

        public LightElementInstance(LightHTMLElementFlyweight flyweight) => _flyweight =
flyweight;
        public void AddChild(LightNode node) => _children.Add(node);

        public override string OuterHTML()
        {
            StringBuilder sb = new StringBuilder();
            sb.Append($"<{_flyweight.TagName}>");
            foreach (var child in _children) sb.Append(child.OuterHTML());
            sb.Append($"</{_flyweight.TagName}>");
            return sb.ToString();
        }
    }

    public class LightElementNode : LightNode
    {
        private string _tagName;
        private List<LightNode> _children = new List<LightNode>();

        public LightElementNode(string tagName) => _tagName = tagName;
        public void AddChild(LightNode node) => _children.Add(node);

        public override string OuterHTML()
        {
            StringBuilder sb = new StringBuilder();
            sb.Append($"<{_tagName}>");
            foreach (var child in _children) sb.Append(child.OuterHTML());
            sb.Append($"</{_tagName}>");
            return sb.ToString();
        }
    }
}
```

```
D:\Lab\kpz_lab3.6\kpz_lab3.6\

Результат верстки книги
<body><h1>ROMEO AND JULIET</h1><h2>ACT V</h2><blockquote> Scene I. Mantua. A Street.</blockquote><p>The
 vault is opened and Romeo enters the tomb to find Juliet.</p></body>

Легковаговик
Унікальних об'єктів тегів у пам'яті: 4
Зайнято пам'яті для дерева: 1944 байт
```

Рис.7.Результат виконання програми: